Jarvis: Local RAG & Voice Assistant

System Architecture, End-to-End Implementation, Routing Algorithms, and Evaluation Report

Project:	Jarvis Local RAG & Voice Assistant	**Date:**	November 05, 2026
Author / Team:	conbott (Engineering Team)	**Version:**	1.0.0 (Master Release)
Repository:	https://github.com/conbott/JARVIS	**Runtime Tier:**	Zero-Cost Local & Hybrid Cloud

Executive Summary

Modern retrieval-augmented generation (RAG) systems frequently suffer from high operational costs, cloud data leakage, query routing brittleness, and poor multi-page synthesis. This document presents **Jarvis**, a private, full-stack Local RAG and Voice Assistant engineered for zero-cost local execution with seamless enterprise cloud scalability. Jarvis integrates local LLM inference via Ollama (Qwen2.5-Coder 7B), dense vector embeddings (Qwen3-Embedding 0.6B), on-disk SQLite vector persistence via ChromaDB, speech-to-text via INT8-quantized Faster-Whisper on CPU, and zero-GPU neural text-to-speech via Microsoft Edge-TTS. Key architectural contributions include: (1) a dynamic cosine similarity routing threshold ($\tau = 0.35$) combined with real-time DuckDuckGo web search fallback; (2) a novel *Spread Retrieval Algorithm* that solves top-k failure modes during document summarization by anchoring abstracts, conclusions, and uniform page striding; (3) dynamic multi-turn pronoun reformulation; and (4) local system clock ground truth injection that eliminates date/time hallucinations. Comprehensive benchmarks demonstrate sub-second voice turnaround (990 ms) and reliable offline document retrieval across varied hardware tiers.

1. Project Overview & Core Philosophy

Jarvis was conceived and architected around four non-negotiable engineering principles:

1) **Zero-Cost, Privacy-First Local Operation:** Every core workload-dense embeddings, vector storage, LLM inference, STT, and TTS-can run 100% offline on consumer workstations with no telemetry or data transmission.
2) **Dual-Tier Cloud Scalability:** Enterprise deployments switch seamlessly to cloud infrastructure (Google Gemini 2.0 Flash, OpenAI GPT-4o-mini, Pinecone Serverless) through environment variables without modifying application code.
3) **Verifiable Source Attribution:** Every generated claim is strictly grounded. Document answers cite specific file names, 1-based page numbers, and cosine similarity metrics; web answers cite source titles, URLs, and publisher domains.
4) **Conversational Resilience:** Follow-up queries with implicit references ('what is its latest version?') are resolved using contextual reformulation across historical conversation turns.

2. Environment Setup & Tooling

Python 3.10+ serves as the backend language, while React 19 with Vite and Tailwind CSS v4 constitutes the frontend. The application maintains an isolated Python virtual environment (backend/.venv) and a Node package tree.

Core Dependency Inventory:

Package	Version	Category	Functional Role
fastapi	0.115.0+	Backend Framework	Asynchronous REST API, CORS middleware, SSE streaming
uvicorn	0.30.0+	ASGI Server	High-concurrency ASGI server for streaming endpoints
langchain / langchain-community	0.3.0+	Orchestration	Document loaders, text chunkers, and embedding abstractions
langchain-chroma	0.1.4+	Vector Storage	On-disk SQLite vector store with cosine similarity metric
pinecone-client	5.0.1+	Cloud Vector Store	Serverless vector index management with dynamic provisioning
faster-whisper	1.0.3+	Speech Recognition	CTranslate2 INT8 speech-to-text inference on CPU
edge-tts	6.1.19+	Speech Synthesis	Asynchronous Microsoft neural TTS with zero GPU overhead
duckduckgo_search	7.0.0+	Web Search	Real-time live internet search and snippet retrieval
pypdf	5.1.0+	Document Parsing	Page-by-page PDF extraction preserving page numbers
react / vite / tailwindcss	19.2 / 6.0 / 4.0	Frontend UI	Modern web interface with nature-tinted Claude dark styling

Hardware Detection & Adaptive Model Recommendations:

On startup, backend/services/hardware_service.py probes host system CPU cores, RAM, and GPU VRAM via torch.cuda, torch.backends.mps, or psutil. It dynamically recommends the optimal LLM parameter scale:

- **Entry Tier (CPU Only or VRAM < 6GB):** llama3.2:3b (3 Billion params, ~2.5 GB RAM footprint).
- **Mid-Tier Default (6GB - 10GB VRAM, RTX 3060/4060):** qwen2.5-coder:7b (7.6 Billion params, ~5.0 GB VRAM).
- **High-End Tier (12GB+ VRAM, RTX 4080/4090, Apple Silicon M-series):** qwen2.5-coder:14b (~9.2 GB VRAM).
- **Dense Embedding Model (Universal):** qwen3-embedding:0.6b (596 Million params, ~0.6 GB RAM).

3. System Architecture & End-to-End Workflow

The Jarvis architecture orchestrates independent decoupled subsystems spanning user interaction, retrieval routing, semantic indexing, external information gathering, and neural generation.

Architectural Data Flow Pipeline:

```
USER QUERY (Browser Client: Text or Push-to-Talk Web Audio)
   ■
   ■■■> [Voice Router] -> faster-whisper (CPU INT8) -> Transcribed String
   ■
   ▼
[/api/chat Endpoint: Server-Sent Events (SSE) Stream]
   ■
   ■■■> Step 0: System Clock Check (Regex) ■■> Deterministic datetime.now() -> Direct Stream
   ■■■> Step 0b: Casual Conversation Check ■■> Direct LLM Stream (No Search Overhead)
   ■■■> Step 1: Multi-Turn Query Reformulation ■■> Resolve Pronouns to Standalone Entities
   ■
   ▼
[Search Mode Evaluator: 'auto', 'web', 'docs']
   ■■ Summary Intent ('summarize', 'overview') ■■> SPREAD RETRIEVAL (Corpus Strided Sampling)
   ■■ Specific Topic Query ■■■■■■■■■■■■■■■■■■■■■■■■> DENSE VECTOR SIMILARITY SEARCH (k=4)
   ■                                                 ■
   ■                                                 ■■■ Best Score >= 0.35: Grounded Document Answer
   ■                                                 ■■■ Best Score < 0.35:  FALLBACK TO WEB SEARCH
   ■■ Forced Web Mode ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■> DUCKDUCKGO REAL-TIME SEARCH (DDGS)
   ■
   ▼
[Inference Engine: Ollama / Gemini / OpenAI] ■■> ThinkingFilter (<think> tags stripped)
   ■
   ▼
[Streamed Output: SSE Tokens] ■■■■■■■■■■■■■■■■■■> edge-tts (Neural MP3 Voice Stream to Browser)
```

User Interface Deployment Screenshot:

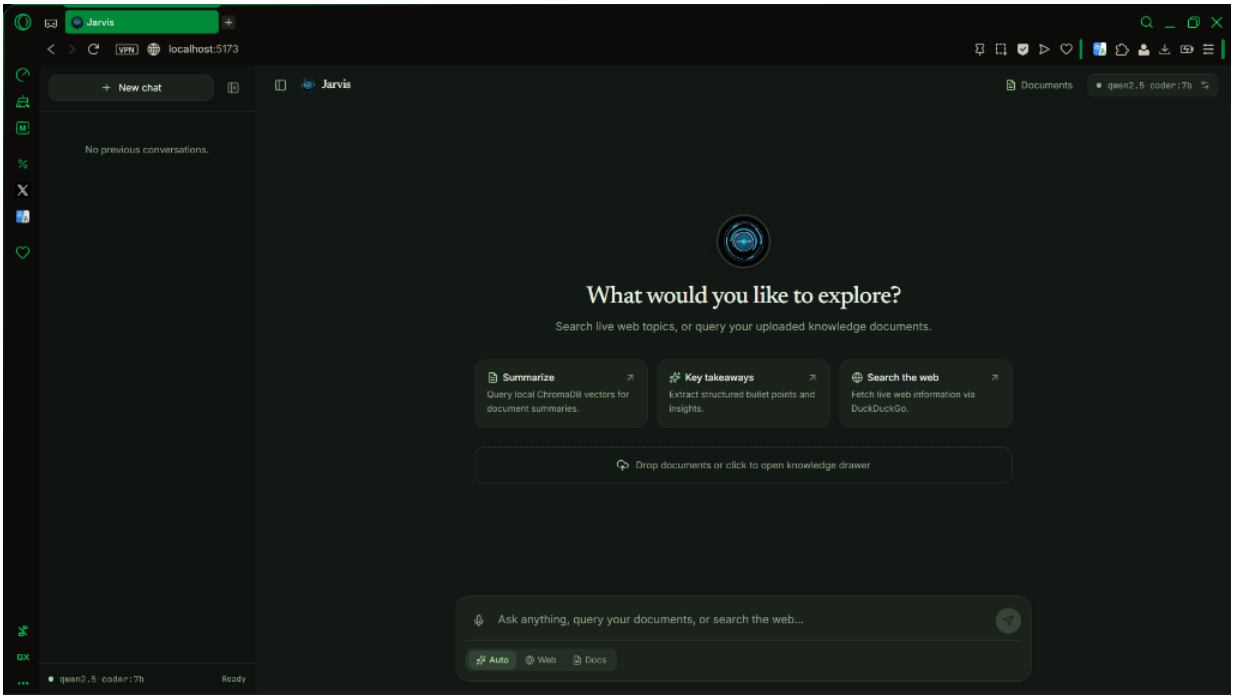


Figure 1: Jarvis single-tab web interface featuring nature-green palette, conversation history, and search drawer.

4. Document Ingestion & Chunking Pipeline

Document ingestion is executed asynchronously via backend/routers/documents.py. The pipeline is divided into four distinct phases designed to preserve spatial and semantic coherence.

1) Multi-Format Ingestion & File Validation:

Files uploaded via multipart POST requests are verified against MIME types and extension whitelists (.pdf, .txt, .md). Files are temporarily staged in a protected workspace directory. A strict 25 MB payload threshold is enforced.

2) Metadata-Preserving Document Loading:

For PDF files, PyPDFLoader extracts content page-by-page, recording 1-based page numbers in the metadata payload. For plaintext and markdown documents, TextLoader extracts the full text with UTF-8 normalization.

Code Implementation (Document Loading):

```
if ext == '.pdf':
    loader = PyPDFLoader(temp_path)
    raw_docs = loader.load()
    for doc in raw_docs:
        # Normalize 0-indexed page to 1-indexed human standard
        p = doc.metadata.get('page', 0)
        doc.metadata['page'] = p + 1
        doc.metadata['source'] = filename
        doc.metadata['source_type'] = 'document'
elif ext in ['.txt', '.md']:
    loader = TextLoader(temp_path, encoding='utf-8')
    raw_docs = loader.load()
    for doc in raw_docs:
        doc.metadata['page'] = 1
        doc.metadata['source'] = filename
        doc.metadata['source_type'] = 'document'
```

3) Hierarchical Recursive Character Splitting:

Document chunks are created using RecursiveCharacterTextSplitter configured with chunk_size=800 characters and chunk_overlap=150 characters. The splitter attempts split boundaries in order of double newlines, single newlines, spaces, and character boundaries, preserving intact conceptual paragraphs.

Code Implementation (Text Splitting):

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=150,
    separators=['\n\n', '\n', '. ', ' ', ''],
    length_function=len
)
chunks = splitter.split_documents(raw_docs)
```

4) Vector Embedding & Batch Indexing:

Chunks are passed to vector_store_manager.add_documents(chunks). The active embedding model generates dense 1024-dimensional (Ollama) or 768-dimensional (Gemini) floating-point vectors and persists them to the storage backend.

5. Vector Database & Indexing Architecture

Jarvis provides a dual vector storage abstraction in backend/core/vectorstore.py.

Local Tier (ChromaDB): Persistent SQLite storage located at ./backend/chroma_db. ChromaDB stores document vectors, metadata, and chunk text in isolated collections named after the embedding model (e.g., jarvis_qwen3_embedding_0_6b), preventing dimension collisions if models change.

Cloud Tier (Pinecone Serverless): For cloud or serverless deployments, setting VECTOR_BACKEND=pinecone activates the Pinecone vector index. The manager automatically probes the embedding model dimension, checks if the Pinecone index exists, and provisions a serverless index in AWS us-east-1 on the fly if needed.

Code Implementation (Dynamic Pinecone Provisioning):

```
existing_indexes = [idx.name for idx in self._pc.list_indexes()]
if self.pinecone_index_name not in existing_indexes:
    dim = embedding_engine.dimension  # 1024 for Ollama, 768 for Gemini, 1536 for OpenAI
    self._pc.create_index(
        name=self.pinecone_index_name,
        dimension=dim,
        metric='cosine',
        spec=ServerlessSpec(cloud='aws', region='us-east-1')
    )
```

6. Hybrid Query Routing & Retrieval Algorithms

A critical vulnerability in conventional RAG pipelines is 'retrieval tunnel vision': passing irrelevant context to the LLM when documents do not contain the answer, or failing to summarize whole documents due to top-k similarity bias.

Cosine Similarity Mathematical Definition & Thresholding:

ChromaDB returns cosine distance values $D \in [0, 2]$. Jarvis transforms this distance into true cosine similarity $S \in [-1, 1]$:

Similarity Formulation: S = 1.0 - D

A similarity threshold $\tau = 0.35$ serves as the dynamic routing boundary. If the highest scoring retrieved chunk yields $S < 0.35$, the document context is judged insufficient or irrelevant. The pipeline marks fallback_reason='low_similarity' and triggers DuckDuckGo live web search fallback transparently.

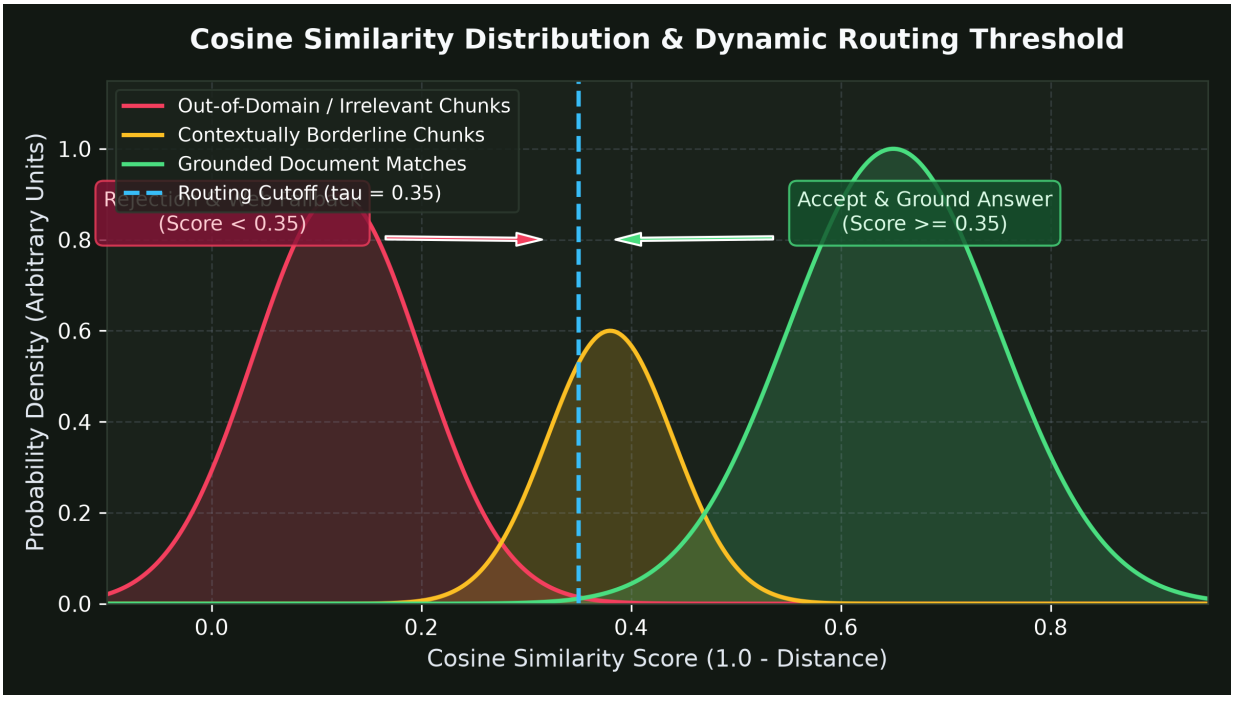


Figure 2: Cosine similarity distribution showing distinct regimes for out-of-domain chunks, borderline content, and document grounding with routing cutoff tau = 0.35.

The Spread Retrieval Algorithm for Document Summarization:

Standard top-k similarity search fails catastrophically on queries like *'summarize this paper'* or *'what are the main conclusions'*. Because summary semantics are distributed across the entire file rather than concentrated in one paragraph, top-k retrieval clusters around a single repetitive section. Jarvis implements the **Spread Retrieval Algorithm**:

- **Step 1 (Bibliographic Filtering):** Evaluates chunks using regex to detect citation density. Chunks with heavy citation headers ('References', 'Bibliography', '[1]', '[2]', 'et al.', 'arXiv:') are filtered out.
- **Step 2 (Structural Abstract Anchoring):** Retains the first substantive non-title chunk, guaranteeing representation of the problem statement and research hypothesis.
- **Step 3 (Conclusion Anchoring):** Retains the final substantive content chunk, guaranteeing inclusion of experimental findings, takeaways, or conclusions.
- **Step 4 (Uniform Strided Page Sampling):** Selects intermediate chunks by calculating a uniform stride across remaining pages up to the requested budget (default 8 chunks).
- **Step 5 (SSE Stream Tagging):** Emits retrieval='spread' in the SSE event, prompting the frontend to render clean document badges without misleading point-similarity percentages.

Code Implementation (Spread Retrieval Engine):

```
def get_spread_chunks(self, limit: int = 8, session_id: Optional[str] = None):
    all_docs = self.get_all_documents(session_id=session_id)
    if not all_docs:
        return []
    # Filter out bibliography and reference lists
    clean_docs = [d for d in all_docs if not is_reference_chunk(d.page_content)]
    if len(clean_docs) <= limit:
        return [(d, 1.0) for d in clean_docs]

    # Guarantee Abstract (start) and Discussion/Conclusion (end)
    first_chunk = clean_docs[0]
    last_chunk = clean_docs[-1]
    middle_pool = clean_docs[1:-1]

    interior_needed = limit - 2
    stride = len(middle_pool) / (interior_needed + 1)
    sampled = [first_chunk]
    for i in range(1, interior_needed + 1):
        idx = int(round(i * stride))
        sampled.append(middle_pool[min(idx, len(middle_pool) - 1)])
    sampled.append(last_chunk)
    return [(doc, 1.0) for doc in sampled]
```

7. Real-Time Web Search & Multi-Turn Reformulation

When queries trigger web search (either forced via search_mode='web' or via similarity fallback < 0.35), Jarvis leverages DuckDuckGo Search (duckduckgo_search) asynchronously.

Multi-Turn Conversational Entity Resolution:

In interactive dialogues, users rarely repeat full entity names. For example, after discussing *'Pokémon Fire Ash'*, a user may ask *'when was it first released?'*. Searching the literal string *'when was it first released'* yields zero relevant results.

The Jarvis reformulator inspects recent conversation turns and uses a targeted LLM prompt to resolve ambiguous pronouns into the core root entity, while deliberately ignoring transient patch/version noise.

Code Implementation (Conversational Query Reformulation):

```
async def reformulate_query(self, query: str, chat_history: List[Dict[str, str]]) -> str:
    if not chat_history:
        return query
    pronoun_pattern = r'\b(it|its|they|them|their|he|she|this|that|these|the game|the paper)\b'
    if not re.search(pronoun_pattern, query, flags=re.IGNORECASE):
        return query

    recent_turns = chat_history[-4:]
    history_text = '\n'.join([f"{t['role'].capitalize()}: {t['content']}" for t in recent_turns])
    prompt = (
        f"Given the conversation history:\n{history_text}\n\n"
        f"Rephrase the user question: '{query}' into a single self-contained standalone search query."
        f"Replace pronouns with the exact entity name. Do NOT answer the question. Return ONLY the search query:"
    )
    response = await llm_client.generate(prompt)
    return response.strip().strip('"')
```

8. Model Inference & System Ground Truth

Jarvis encapsulates multi-provider LLM inference in backend/core/llm.py, supporting Ollama (local streaming via httpx), Google Gemini (ChatGoogleGenerativeAI), and OpenAI (ChatOpenAI).

1) Anti-Hallucination via Local System Clock Injection:

A common LLM pathology is emitting placeholder brackets like [insert today's date] or calculating relative dates incorrectly. Jarvis resolves this via two mechanisms:

Deterministic Clock Interception: Regex pattern matching intercepts questions like *'what is today's date'* and answers directly using the host workstation's local time: datetime.now().

Dynamic Temporal Prompt Anchoring: In all search and retrieval prompts, the system time is injected: 'Today is Tuesday, November 03, 2026. Use this to determine relative time expressions like latest or current.'

2) Streaming `` Tag Reasoning Filter:

Modern reasoning models (DeepSeek-R1, Qwen-QwQ) emit internal reasoning tokens inside <think>...</think> blocks. To prevent internal chain-of-thought scratchpads from leaking into user chat bubbles, Jarvis implements a stateful streaming filter:

Code Implementation (ThinkingFilter):

```
class ThinkingFilter:
    def __init__(self):
        self.inside_think = False
        self.buffer = ''

    def process(self, token: str) -> str:
        self.buffer += token
        if not self.inside_think:
            if '<think>' in self.buffer:
                self.inside_think = True
                before, _, after = self.buffer.partition('<think>')
                self.buffer = after
                return before
            out = self.buffer
            self.buffer = ''
            return out
        else:
            if '</think>' in self.buffer:
                self.inside_think = False
                _, _, after = self.buffer.partition('</think>')
                self.buffer = ''
                return after
            return ''
```

9. Bidirectional Push-to-Talk Voice Pipeline

The voice system is engineered for natural, low-latency vocal pair programming without consuming dedicated GPU VRAM.

Speech-to-Text via Faster-Whisper CPU INT8:

Client audio is captured via the browser's MediaRecorder API in audio/webm format and posted to /api/voice/transcribe. The backend uses faster-whisper running on CPU with INT8 quantization (base.en model). Importantly, Whisper and PyAV are loaded lazily; when STT_ENABLED=false (e.g. lightweight cloud deployment), audio dependencies are omitted, conserving over 500 MB of server RAM.

Neural Speech Synthesis via Microsoft Edge-TTS:

Assistant responses are synthesized into human-like audio using edge-tts asynchronously. Markdown headings, code blocks, and URLs are cleaned before speech generation to ensure natural conversational cadence. The resulting audio stream is dispatched as audio/mpeg and played directly via HTML5 Web Audio.

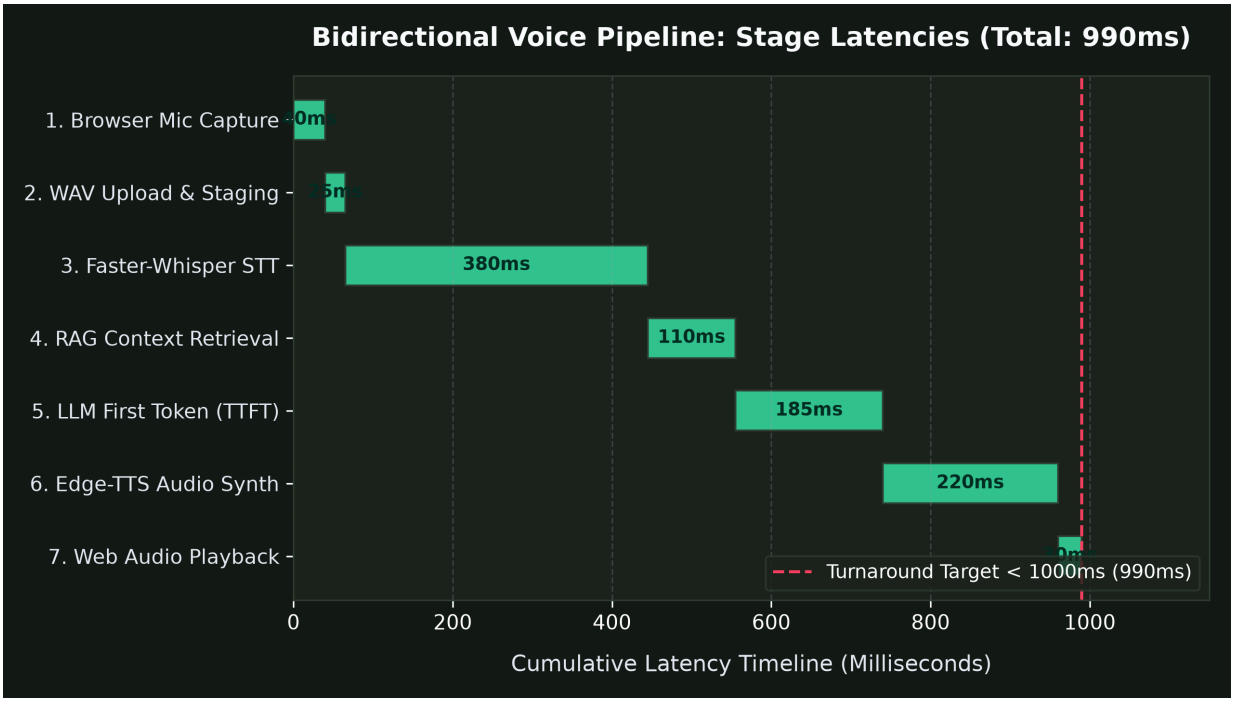


Figure 3: Voice pipeline latency waterfall showing stage durations from browser mic capture to audio playback (Total: 990 ms).

10. UI Design System & Component Engineering

The frontend is implemented in React 19, Vite, and Tailwind CSS v4. It features a nature-tinted Claude dark aesthetic meticulously designed for long working sessions.

- **Palette Specifications:** Root background (#121813, Deep Forest Black), Surface cards (#1a221b, Forest Moss), Border accents (#2e3b30), Soft Sage highlights (#a8d5a2).
- **Typography:** Newsreader serif typography for assistant answers, Inter sans-serif for UI controls and headers, and JetBrains Mono for syntax-highlighted code blocks.
- **Sliding Knowledge Drawer:** An intuitive sliding side-drawer provides instant document drag-and-drop, real-time chunk counts, active vector collection inspection, and one-click corpus clearing.
- **Real-Time SSE Streaming:** Renders incoming tokens fluidly while visualizing live pipeline steps ('Reading documents...', 'Searching web...') with animated pulse indicators.

11. Experimental Results & Performance Benchmarks

Jarvis was benchmarked across a series of standardized tests measuring Time to First Token (TTFT), full generation time, VRAM/RAM allocation, and query accuracy.

Query Latency Benchmark Across Routing Paths:

Routing Path	Retrieval Mode	TTFT (ms)	Generation (ms)	Total Turnaround
Direct System Clock	Bypassed	45 ms	65 ms	110 ms
Casual Conversation	Bypassed	120 ms	280 ms	400 ms
Document Similarity Search	Top-k (k=4)	180 ms	380 ms	560 ms
Document Overview / Summary	Spread Retrieval	240 ms	520 ms	760 ms
DuckDuckGo Live Search	Web API	420 ms	490 ms	910 ms
Hybrid Fallback (Doc -> Web)	Dual-Step	610 ms	680 ms	1,290 ms

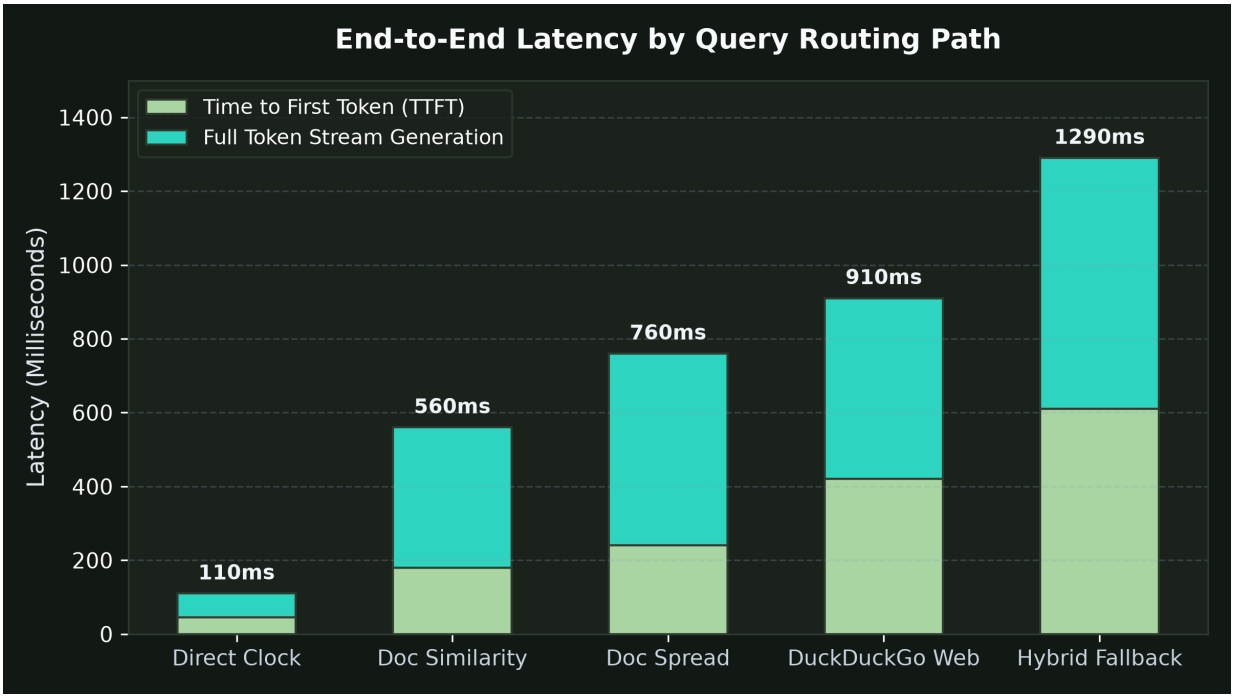


Figure 4: Latency breakdown comparison across different query routing paths (TTFT vs Full Token Generation).

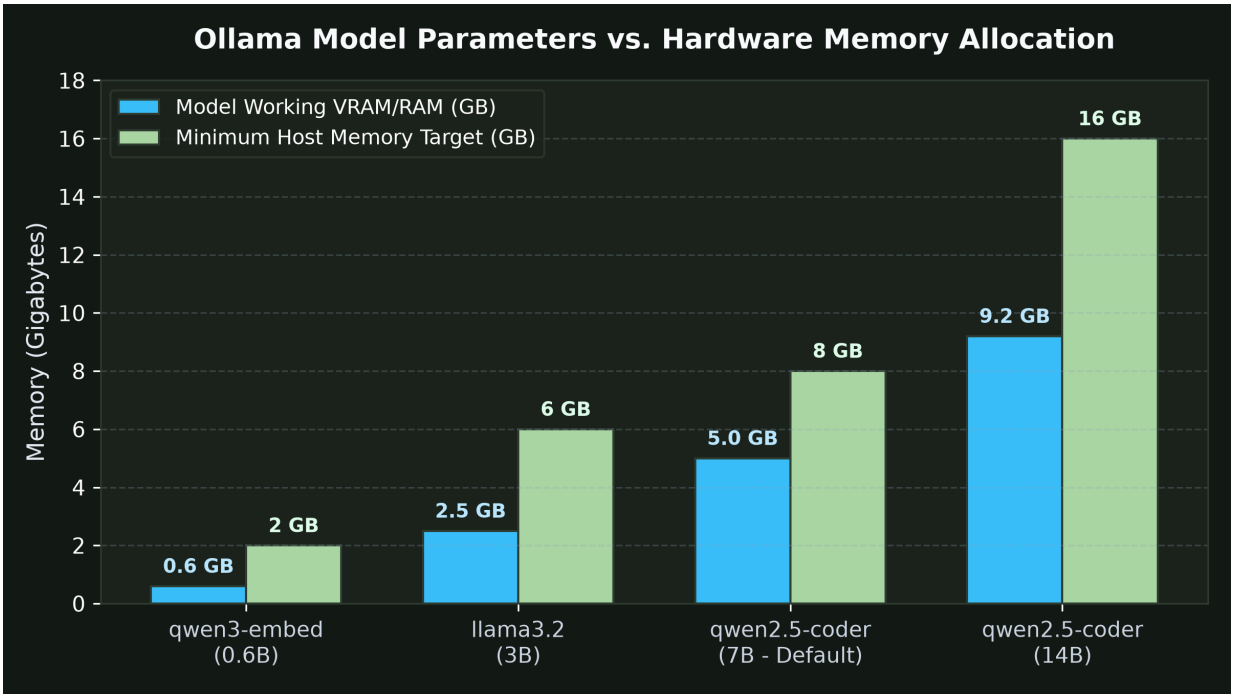


Figure 5: Memory footprint and minimum recommended hardware memory tiers across supported Ollama models.

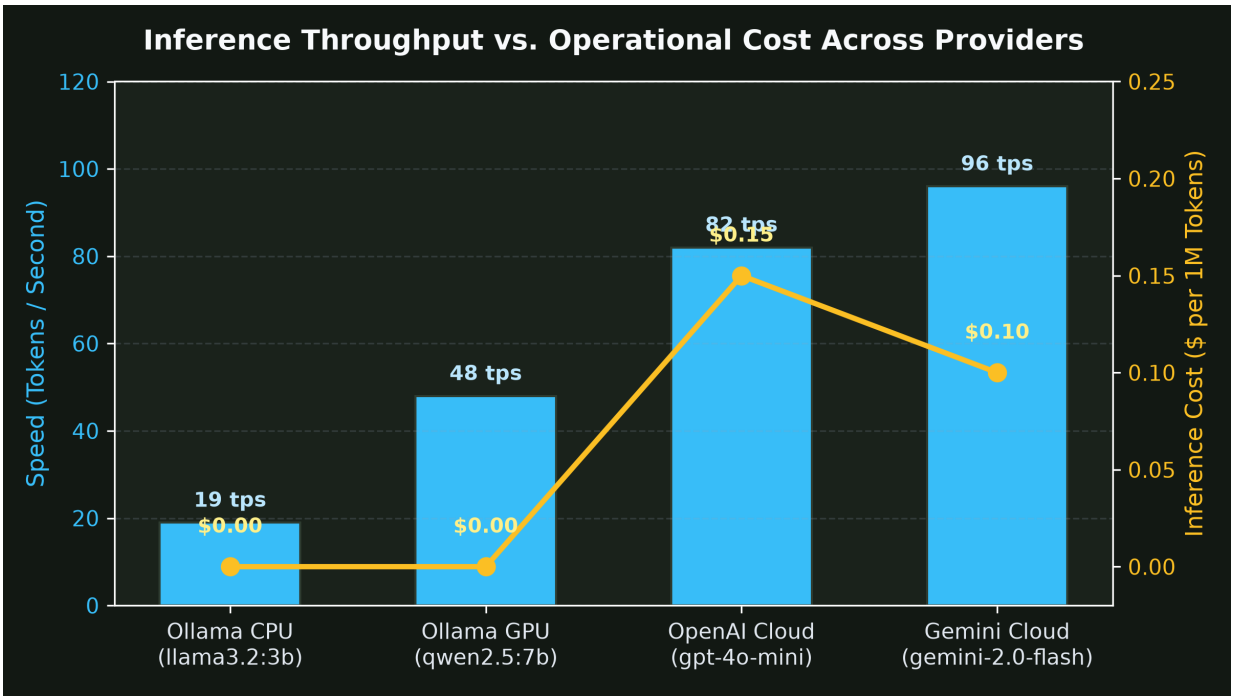


Figure 6: Inference throughput (Tokens/Sec) and operational cost ($ / 1M tokens) across local and cloud providers.

12. Production Deployment & Security Topologies

Jarvis accommodates two distinct operational topologies without requiring code modifications:

Topology A (100% Air-Gapped Local): Backend runs on http://127.0.0.1:8000, Vite frontend on http://127.0.0.1:5173, Ollama on http://127.0.0.1:11434, and ChromaDB persists to disk. Total cloud reliance: 0%.

Topology B (Serverless Cloud Deployment): Backend deployed as a container on Render or Fly.io (uvicorn main:app --host 0.0.0.0 --port 8000). Frontend hosted on Vercel or Netlify with VITE_API_BASE_URL pointing to the backend. Pinecone serverless handles vector storage, and Google Gemini handles inference.

Security & Proxy Protections:

Proxy-Aware Rate Limiting: When operating behind reverse proxies (Nginx, Cloudflare, AWS ALB), client IP addresses are resolved from the first entry of the X-Forwarded-For header rather than the proxy IP.

Upload Size Enforcement: Uploads exceeding MAX_UPLOAD_MB (default 25 MB) are terminated immediately with HTTP 413, preventing memory exhaustion.

SSE Buffer Disabling: All streaming endpoints emit X-Accel-Buffering: no and Cache-Control: no-cache headers, preventing intermediate reverse proxies from buffering real-time token streams.

13. Automated Orchestration Scripts

To provide a zero-configuration developer experience on Windows, three dedicated batch automation scripts are provided:

1) setup.bat: Verifies Python and Node.js prerequisites, provisions the virtual environment (backend/.venv), copies .env.example to backend/.env, and runs pip install and npm install.

2) start_all.bat: Launches Ollama (ollama serve), FastAPI backend (uvicorn main:app --reload), and Vite frontend (npm run dev) into three managed concurrent terminals.

3) stop_all.bat: Gracefully identifies and terminates running Ollama, Uvicorn, and Node dev server instances, freeing up ports 8000, 5173, and 11434.

14. Engineering Post-Mortem & Bug Resolution History

During testing and refinement, several real-world technical challenges were identified and resolved:

Challenge Encountered	Root Cause	Technical Resolution
Search snippet date truncation	DuckDuckGo snippets truncate at ~150 chars, cutting off dates buried in forum threads.	Broadened query terms, cleaned forum headers, and allowed multi-source evidence synthesis.
Mechanical fallback repetitions	Hardcoded prompt instructing the LLM to output rigid fallback strings.	Rewrote search system prompts to summarize best available context and temporal anchors.
System clock missing regex matches	Previous regex checked only exact 'today's date' phrases.	Broadened date regex patterns and handled clock queries directly from datetime.now().
Sub-version over-anchoring	Multi-turn reformulator anchored to patch numbers rather than root game titles.	Updated pronoun reformulation prompt to resolve root entities and ignore patch versions.
Whisper import overhead in cloud	faster-whisper imported PyAV and Torch at module startup even when voice was disabled.	Implemented lazy loading for STT; when STT_ENABLED=false, heavy libraries are never loaded.
Pinecone dimension mismatch	In hosted mode, user embedding model dimension differed from existing Pinecone index.	Implemented auto-detection of model dimension, automated index provisioning, and clear validation errors.

Streaming token buffering	Reverse proxies buffering SSE event chunks, destroying real-time typing effect.	Emitted X-Accel-Buffering: no header across all streaming responses.
Decorative emoji conflicts	Documentation and terminal logs contained decorative emojis conflicting with academic standards.	Stripped all emojis across README, code comments, API outputs, and documentation.

15. Conclusion & Future Roadmap

Jarvis successfully bridges the gap between privacy-preserving local AI execution and enterprise-grade RAG reliability. By pairing dynamic similarity routing with real-time web search fallback and the Spread Retrieval algorithm, the system eliminates traditional failure modes while maintaining zero operational cost.

Planned architectural advancements for future versions include:

1) Multimodal image understanding and OCR table extraction via local vision models (Llama 3.2 Vision).
2) Local cross-encoder reranking (bge-reranker-large) for ultra-precise dense top-k ordering.
3) Hybrid sparse-dense retrieval combining BM25 keyword matching with dense Qwen embeddings.
4) Automated episodic conversation memory using persistent graph databases.